

Implementation Observations and Discrepancies

Companion to the HLD and LLD

Field	Value
Project Name	Loan Application and Compliance Review Assistant
Document Type	Implementation Observations and Documentation Discrepancies
Version	1.0
Date	22 September 2026
Companion Documents	<code>HLD-Loan-Application-and-Compliance-Review-Assistant.md</code> , <code>LLD-Loan-Application-and-Compliance-Review-Assistant.md</code>
Method	Full source inspection, EF Core model and migration review, configuration review, and execution of all six test suites
Build Result	<code>dotnet build LoanAssistant.slnx</code> — succeeded, 0 errors, 0 warnings
Test Result	161 passed, 0 failed, 0 skipped
Code Changes Made	None. This is a documentation exercise only.

Purpose of this document. While writing the HLD and LLD, every significant statement was cross-checked against the repository. This document records each place where the code differs from earlier project documentation, where a component exists but is not reached on a live path, or where the implementation contains a defect or a limitation worth knowing about.

Nothing here was fixed. Each item is reported as an observation, with its evidence and its practical impact, so you can decide what — if anything — to act on.

How to read this document

Items are grouped by kind and each carries a severity, meaning:

Severity	Meaning
Defect	The code does not do what it appears intended to do.
Doc mismatch	The code is fine; earlier documentation describes it inaccurately.
Dormant	The component is built and registered but not reached on any live request path.
Limitation	Working as written, but with a constraint that should be stated rather than glossed over.
Security note	Worth knowing before this pattern goes anywhere near production.

Summary table

#	Observation	Kind	Priority
1	Two live recommendation paths coexist	Limitation	Medium
2	Semantic Kernel is registered but never invoked	Dormant	Medium
3	MCP endpoint has no authorisation attribute	Security note	High
4	Policy frontmatter key casing mismatch	Defect	High
5	Admin dashboard status tiles are hardcoded	Doc mismatch	Medium
6	Document extraction is text-only — no OCR	Limitation	High
7	<code>DocumentProcessingWorker</code> is a stub	Doc mismatch	Medium
8	Test counts and milestones drift in <code>implementation-status.md</code>	Doc mismatch	Low
9	Domain state restored by reflection	Limitation	Medium
10	<code>ExtractedField<T></code> is dead code	Limitation	Low
11	<code>SqlRecommendationRepository.SaveAsync</code> derives a wrong application ID	Defect	Medium
12	MCP tool names differ from README	Doc mismatch	Medium
13	MCP decision refusal is not a <code>-32602</code> error	Doc mismatch	Low
14	<code>ResiliencePolicy</code> is not registered in DI	Limitation	Low
15	<code>appsettings.json</code> has no <code>AzureAISearch</code> section	Limitation	Medium
16	"CQRS" is a command/query handler pattern	Doc mismatch	Low
17	Unused injected dependency in <code>OfficerController</code>	Limitation	Low
18	Compliance security events are illustrative, not real	Doc mismatch	Medium
19	Identity threshold differs between two code paths	Limitation	Low

#	Observation	Kind	Priority
20	Search filter values are string-interpolated	Security note	Low
21	Unreachable <code>Views/Home/Index.cshtml</code>	Limitation	Low
22	No deployment or CI/CD artefacts exist	Doc mismatch	Medium

A. Architecture and design observations

1. Two live recommendation paths coexist

Kind: Limitation · Priority: Medium

What was found. The system contains two separate recommendation generators. Both are active and both write to the same `Recommendations` table.

	Path A	Path B
Component	<code>GenerateRecommendationDraftCommandHandler</code>	<code>RecommendationOrchestratorAgent</code> → <code>SaveRecommendationDraftCommandHandler</code>
Triggered from	<code>ApplicantController.CreateApplication</code> , <code>.UploadDocument</code> , <code>.ConfirmField</code> ; <code>DemoDataSeeder</code>	<code>OfficerController.Review</code> , <code>.StreamRecommendationDraft</code>
AI used	None	Four <code>gpt-4o</code> calls
Retrieval used	None	Yes
Citations	One hardcoded: <code>"Mortgage Eligibility Standard"</code> , section 4.1	Real, from retrieved passages

Evidence. `src/Loan.Application/Recommendations/RecommendationCommands.cs` builds the citation list as a literal. `src/Loan.Web/Controllers/ApplicantController.cs` calls the deterministic handler at three points; `src/Loan.Web/Controllers/OfficerController.cs` calls the orchestrator.

Impact. Earlier documentation presents the multi-agent flow as *the* recommendation flow. In practice an applicant sees a deterministic draft with a placeholder citation immediately after submitting, and that draft is replaced by the multi-agent draft only when an officer opens the application. Because `GetByIdAsync` selects the latest recommendation by descending identifier — and identifiers are random GUID fragments rather than sequential — which draft is shown is not strictly guaranteed to be the newest by time.

Why it is probably intentional. Path A gives an applicant instant feedback without incurring model cost on every upload. That is a reasonable design; it just needs stating.

Both documents describe both paths and where each triggers.

2. Semantic Kernel is registered but never invoked

Kind: Dormant · Priority: Medium

What was found. `Microsoft.SemanticKernel` v1.34.0 is referenced. `SemanticKernelAgentService` is registered as `Scoped` , and four plugins carrying `[KernelFunction]` attributes are registered alongside it: `IdentityPlugin` , `CreditPlugin` , `PolicySearchPlugin` , `DraftSaverPlugin` .

Evidence. A solution-wide search for `SemanticKernelAgentService` and `RunAgentPromptAsync` returns exactly three hits: the class itself, its DI registration in `src/Loan.Infrastructure/DependencyInjection.cs:62` , and `tests/Loan.IntegrationTests/SemanticKernelTests.cs` . **No controller, handler or agent calls it.**

Impact. The AI orchestration that actually runs is the custom `RecommendationOrchestratorAgent`, which calls `IChatModel` directly. Semantic Kernel is configured, instantiated on request, and test-covered — but contributes nothing to any live request. Anyone reading the package list or the DI registration could reasonably conclude otherwise.

Note also. The plugins pass an **application identifier** where the verification services expect a **synthetic identifier** — for example `IdentityPlugin.GetIdentityStatusAsync(applicationId)` calls `VerifyIdentityAsync(applicationId)`. Since synthetic services key on `SYN-...` values, these plugin calls would return "not found" if they were ever invoked.

Both documents mark Semantic Kernel as integrated and test-covered but not on a live path.

3. MCP endpoint has no authorisation attribute

Kind: Security note · Priority: High

What was found. `McpController` carries `[ApiController]` and `[Route("api/mcp")]` but no `[Authorize]` attribute, so `POST /api/mcp` accepts anonymous requests.

Evidence. `src/Loan.Web/Controllers/McpController.cs:8-9`. Actor resolution falls through to `actorId ??= "SystemWorker"` and `actorRole ??= "SystemWorker"`. In Development — or when an `X-MCP-Test-Key` header is present — the caller-supplied `X-Actor-Id` and `X-Actor-Role` headers are honoured.

What still protects the endpoint. Three things, all intact:

- `ValidateApplicationScopeAsync` rejects a synthetic identity that does not belong to the named application, so cross-applicant reads are blocked.
- `save_draft` refuses `Approve / Reject` routing states, and `SaveRecommendationDraftCommandHandler` refuses them again independently.
- The `SystemWorker` default role cannot make officer decisions.

Residual exposure. An unauthenticated caller who can reach the endpoint can enumerate the tool catalogue and read synthetic verification records for any application whose identifier and synthetic identifier they know or can guess. Since all data is synthetic, the practical harm in this build is low; the pattern would be unacceptable in production.

Both documents record this in their security sections.

B. Defects found in the implementation

4. Policy frontmatter key casing mismatch

Kind: Defect · Priority: High

What was found. The administrator policy upload path and the indexer's parser disagree on frontmatter key casing, so administrator-uploaded documents are indexed with default metadata.

`AdminController.UploadPolicyDocument` writes (snake_case):

```
document_id:, title:, product_id:, policy_version:,  
document_type:, audience:, effective_from:, effective_to:
```

`PolicyIndexer.ParsePolicyMarkdown` reads (PascalCase) — and only these:

```
DocumentId, Title, ProductId, PolicyVersion,  
DocumentType, Audience, EffectiveFrom, EffectiveTo
```

Evidence. `src/Loan.Web/Controllers/AdminController.cs` (frontmatter generation) versus the `switch (key)` block in `src/Loan.Infrastructure/Search/PolicyIndexer.cs`.

Impact. When an administrator uploads a policy file that has **no frontmatter of its own**, the controller prepends snake_case frontmatter, the parser recognises none of those keys, and every value falls back to its default:

`DocumentId = "DOC-UNKNOWN"`, `title = the filename`, `ProductId = "ALL"`, `PolicyVersion = "v1.0"`, `EffectiveFrom = "2026-01-01"`, `EffectiveTo = "Active"`. Consequences:

- The version and product the administrator typed into the form are discarded.
- The document is indexed as applying to **all** products at version **v1.0**.
- Because the index key is `{DocumentId}-{Section}` and the document identifier is always `DOC-UNKNOWN`, **two different uploaded documents with a same-named section will overwrite each other**, since the indexer uses merge-or-upload on a deterministic key.
- Version-aware retrieval filtering will treat the new document as v1.0 regardless of intent.

Not affected. The eight seed policy documents in `SeedPolicies/` all carry correct PascalCase frontmatter, so the normal indexing path — and every demonstration that uses it — works correctly.

Smallest fix, if you choose to act. Change the eight generated keys in `AdminController` to PascalCase to match the parser. One-line-per-key edit, no parser change needed.

5. Admin dashboard status tiles are hardcoded

Kind: Doc mismatch · Priority: Medium

What was found. The service status values on the Administrator dashboard are literal display strings, not the result of any probe:

```
SqlStatus          = "Connected (LocalDB / MSSQL)",
AzureOpenAiStatus  = "Connected & Active (Managed Credential)",
AzureSearchStatus  = "Connected & Online (loan-policies-index)",
AzureBlobStatus    = "Private Container Active ('loan-documents')",
McpServerStatus    = "Running (POST /api/mcp - Streamable HTTP)",
```

Evidence. `src/Loan.Web/Controllers/AdminController.cs`, the `SystemHealthSummary` construction.

Impact. These tiles read "Connected" whether or not the services are reachable. Two further details worth noting:

- The **telemetry tiles** fall back to fixed numbers when no live telemetry exists for the current correlation identifier — `5392` total tokens, `1787.7` ms average latency, `18` retrieval queries, `12` tool calls. On a freshly-started application these figures appear without any work having been done.
- `AzureOpenAiStatus` says "Managed Credential", but the implementation authenticates with an **API key** (`ApiKeyCredential`), not managed identity.

Where genuine status lives. `GET /health/details` performs real checks and correctly reports `Degraded` when an AI service is unconfigured.

Both documents state that these tiles are presentational and point to `/health/details`.

6. Document extraction is text-only — no OCR

Kind: Limitation · **Priority:** High

What was found. `AzureOpenAiDocumentExtractor` reads the uploaded stream as UTF-8 text:

```
using (var reader = new StreamReader(documentStream, Encoding.UTF8,
    detectEncodingFromByteOrderMarks: true, leaveOpen: true))
{
    documentText = await reader.ReadToEndAsync(cancellationToken);
}
```

Evidence. `src/Loan.Infrastructure/Documents/AzureOpenAiDocumentExtractor.cs`. There is no reference to Azure AI Document Intelligence, Form Recognizer, Tesseract, or any image-to-text capability anywhere in the solution. No vision content parts are sent to the model — only text.

Impact. The upload validator permits `.pdf`, `.jpg`, `.jpeg` and `.png`. Those files are validated, hashed and stored correctly, but produce no usable text, so extraction always falls through to `SyntheticDocumentExtractor`, which returns demonstration values based on the filename. Genuine extraction therefore works only for `.txt`, `.csv`, `.json` and `.md` — which is exactly what the six files in `Loan.Web/SampleDocuments` are.

Terminology. Earlier documentation describes this as "OCR" and "document intelligence", and refers to a "low-confidence OCR gate". The confidence gate is real and works; the mechanism producing the confidence score is LLM text-field extraction, not optical character recognition.

Both documents describe it as LLM-based text-field extraction and state the binary-file consequence explicitly.

7. DocumentProcessingWorker is a stub

Kind: Doc mismatch · Priority: Medium

What was found. The worker's entire loop is:

```
while (!stoppingToken.IsCancellationRequested)
{
    _logger.LogDebug("DocumentProcessingWorker polling queue...");
    await Task.Delay(5000, stoppingToken);
}
```

Evidence. `src/Loan.Workers/Processing/DocumentProcessingWorker.cs` . It injects `IDocumentExtractor` but never calls it. There is no queue, no message broker, no database poll and no extraction work.

Impact. All real document extraction happens synchronously inside `UploadAndExtractDocumentCommandHandler` during the web request. The worker contributes nothing. `docs/implementation-status.md` states "Background processing active" and describes the worker as an "Intake Queue Poller" — it is a placeholder.

By contrast, `PolicyIndexingWorker` does real work: it calls `PolicyIndexer.SynchronizeIndexAndSeedAsync` once at startup, then idles on a 30-minute loop that performs no further indexing.

Both documents mark the document worker as a stub and the indexing worker as functional-at-startup.

8. Test counts and milestones drift in implementation-status.md

Kind: Doc mismatch · Priority: Low

What was found. Three inconsistencies in `docs/implementation-status.md` :

Claim in that document	Verified reality
"100% Passing (147 Tests...)"	161 tests, all passing
Testing Suites row: "34 Tests passing across..."	34 is the <code>Loan.EndToEndTests</code> count alone
Infrastructure row, Next Milestones: "Add Azure AI Search adapter"	<code>AzureAiSearchPolicyRetriever</code> is fully built, registered and live-tested

Verified per-suite counts (from executing each suite):

Suite	Tests
<code>Loan.Domain.Tests</code>	20
<code>Loan.Application.Tests</code>	59
<code>Loan.ContractTests</code>	4
<code>Loan.IntegrationTests</code>	23
<code>Loan.EndToEndTests</code>	34
<code>Loan.PromptTests</code>	21

Suite	Tests
Total	161

`README.md` states 161 and is correct. The LLD uses the verified figures.

C. Code-level observations

9. Domain state restored by reflection

Kind: Limitation · Priority: Medium

What was found. Rehydrating an aggregate from the database uses reflection to set members that the domain model deliberately protects:

- `LoanApplicationEntity.ToDomain` sets `Status` and `UpdatedAtUtc` through `typeof(LoanApplication).GetProperty(...).SetValue(...)`, and populates the **private field** `_documentAuditTrail` via `GetField(..., BindingFlags.NonPublic | BindingFlags.Instance)`.
- `RecommendationEntity.ToDomain` likewise sets `Status`, `ApprovedByOfficerId`, `ApprovedAtUtc` and `OfficerDecisionNotes`.
- `ApplicantController.SaveDraft` sets the `Facts` property by reflection instead of calling `UpdateFacts`.
- `DemoDataSeeder` sets `Status` by reflection in several places.

Evidence. `src/Loan.Infrastructure/Persistence/DbContext/LoanDbContext.cs`, `src/Loan.Web/Controllers/ApplicantController.cs`, `src/Loan.Infrastructure/Persistence/DemoDataSeeder.cs`.

Why it happens. The aggregate exposes private setters and guarded transition methods — correct domain design. But rehydrating an application that is already mid-lifecycle cannot legitimately replay those transitions, so persistence reaches past them.

Impact. Two consequences worth knowing. First, the aggregate's state guards are not exercised on load, so a corrupted `Status` value in the database would be accepted silently. Second, reflection is brittle: renaming `Status` or `_documentAuditTrail` would compile cleanly and fail at runtime with a null-reference dereference on the `!` operator. There is one visible symptom already — `ToDomain` calls `app.EvaluateEligibility(UpdatedAtUtc)` to rebuild indicators, which **recalculates** them from stored facts rather than deserialising the stored `Indicators` JSON, so the persisted indicator values are effectively write-only.

Conventional alternative, if you ever revisit it: give the aggregate an internal rehydration constructor or a static factory for persistence use.

10. `ExtractedField<T>` is dead code

Kind: Limitation · Priority: Low

What was found. `src/Loan.Domain/Applications/ExtractedField.cs` defines a generic `ExtractedField<T>` with its own `Confirm` method and an 0.85 confidence threshold. It is superseded by the non-generic `ExtractedFieldRecord` in `Loan.Domain/Documents/`, which is what every production path uses.

A latent bug inside the dead code. `ExtractedField<T>.Confirm` sets status by comparing the field's own `ConfirmedBy` property — not the incoming parameter — against the literal `"OFFICER"`:

```
Status = ConfirmedBy == "OFFICER" ? ... : ...;    // ConfirmedBy is still null here
ConfirmedBy = confirmedBy;                        // assigned afterwards
```

Since `ConfirmedBy` is assigned on the following line, the comparison always evaluates against `null`, so the status is always `ConfirmedByApplicant`. This never manifests because the type is unused.

Impact. None functionally. It is a maintenance hazard: a future developer could reasonably pick the generic type and inherit the bug.

11. `SqlRecommendationRepository.SaveAsync` derives a wrong application ID

Kind: Defect · Priority: Medium

What was found. The method tries to discover which application a recommendation belongs to by string substitution:

```
var appEntity = await _context.Applications.AsNoTracking()
    .FirstOrDefaultAsync(a => a.ApplicationId.Replace("APP", "REC") == recommendation.RecommendationId
        || a.ApplicationId == recommendation.RecommendationId.Replace("REC", "APP"));

string appId = appEntity?.ApplicationId ?? recommendation.RecommendationId.Replace("REC", "APP");
```

Evidence. `src/Loan.Infrastructure/Persistence/Repositories/SqlRepositories.cs`.

Why it fails. Recommendation identifiers are generated as `$"REC-{Guid.NewGuid().ToString("N")[..8].ToUpperInvariant()}"` — for example `REC-A3F9C1D2`. Applying `Replace("REC", "APP")` produces `APP-A3F9C1D2`, which matches no real application (real ones look like `APP-2026-001`). So the lookup finds nothing and the fallback writes a **non-existent application identifier** into the `Recommendations.ApplicationId` column.

Why it is not visible today. Every caller follows `SaveAsync` immediately with `_applicationRepository.UpdateAsync(app)`, and that method writes the recommendation row again — this time with the **correct** `ApplicationId`, because it is passed explicitly. The correct value therefore overwrites the wrong one within the same operation.

Residual risk. The masking depends entirely on call ordering. Any future path that calls `IRecommendationRepository.SaveAsync` **without** a following application update would persist an orphaned recommendation row that no application query would ever return. Note also that the substitution is naive in general: it would corrupt any identifier containing those letters elsewhere.

Conventional fix, if you act: add the application identifier as a parameter to `IRecommendationRepository.SaveAsync`, since every caller already knows it.

12. MCP tool names differ from README

Kind: Doc mismatch · Priority: Medium

What was found. Three of the five tool names in the README's MCP diagram do not exist in the code.

README diagram	Actual name in <code>GetRegisteredTools()</code>
<code>get_identity_status</code>	<code>get_identity_status</code> ✓
<code>get_income_record</code>	<code>get_income_verification</code>

README diagram	Actual name in <code>GetRegisteredTools()</code>
<code>get_credit_summary</code>	<code>get_credit_score</code> (alias <code>get_credit</code> also accepted)
<code>search_policy_docs</code>	<code>search_policy</code>
<code>save_draft</code>	<code>save_draft</code> ✓

Evidence. `src/Loan.Infrastructure/MCP/McpToolServer.cs`, `GetRegisteredTools()` and the `toolName` switch dispatch.

Impact. Direct and practical: an MCP client built from the README's names would receive `ToolNotFound` for three of the five tools. The LLD uses the verified names.

Related. `AdminController` maps display roles for tool names `officer_decision` and `document_override`, neither of which is a registered tool. That switch arm is unreachable, which suggests the tool set was reshaped during development without the mapping being updated.

13. MCP decision refusal is not a `-32602` error

Kind: Doc mismatch · Priority: Low

What was found. The README states: "Attempts to invoke *Approve or Reject* via MCP return a JSON-RPC `-32602 InvalidParams` error."

The implementation returns a **successful** JSON-RPC response whose `result` is an `McpToolResult` with `isError: true` and an explanatory text message. The JSON-RPC `error` member is null.

Evidence. `McpToolServer.HandleSaveDraftAsync` returns `ErrorResult(...)`, and the `tools/call` case wraps it as `new JsonRpcResponse("2.0", toolResult, null, request.Id)`.

Assessment. The implementation is arguably **more** correct than the documentation: the MCP specification models tool-level failures as `isError` on the tool result, reserving JSON-RPC error codes for protocol-level faults. The code uses `-32600`, `-32601` and `-32603` for genuine protocol faults and never uses `-32602`.

Impact. A client checking for a `-32602` code would not detect the refusal. The LLD documents the actual mechanism.

14. `ResiliencePolicy` is not registered in DI

Kind: Limitation · Priority: Low

What was found. `ResiliencePolicy` is never added to the service collection. Both `AzureOpenAIChatModel` and `AzureAiSearchPolicyRetriever` accept it as an optional constructor parameter and default it:

```
_resiliencePolicy = resiliencePolicy ?? new Resilience.ResiliencePolicy();
```

Impact. Each adapter holds its **own** circuit-breaker state. Since `IChatModel` is a singleton, its circuit is effectively application-wide — that part works as intended. But `IPolicyRetriever` is **scoped**, so a new retriever with a fresh, closed circuit is created per request. The circuit breaker therefore cannot accumulate failures across requests for search, and its protective effect there is limited to retries within a single request. Options are also never configurable — the defaults cannot be tuned from `appsettings.json`.

Minimal fix, if you act: register `ResiliencePolicy` as a singleton (or one named instance per dependency) and bind `ResiliencePolicyOptions` from configuration.

15. `appsettings.json` has no `AzureAISearch` section

Kind: Limitation · **Priority:** Medium

What was found. The tracked `src/Loan.Web/appsettings.json` contains sections for `ConnectionStrings`, `AzureStorage`, `AzureOpenAI`, `Logging` and `AllowedHosts` — but no `AzureAISearch` section at all, despite the retriever and indexer requiring `AzureAISearch:Endpoint`, `AzureAISearch:ApiKey` and `AzureAISearch:IndexName`.

Impact. Nothing is broken on a configured machine: the keys resolve from User Secrets. But someone cloning the repository has no placeholder to discover, so the requirement is invisible until search silently returns empty results. The other services all advertise themselves through placeholders such as `YOUR-RESOURCE-NAME`.

Confirmed on this machine. `dotnet user-secrets list` shows ten keys configured, covering all four services. **No secret values appear in this document or in either design document.** The tracked configuration contains only placeholders — no live key, connection string or password is committed.

Minimal fix, if you act: add a placeholder `AzureAISearch` section to `appsettings.json` alongside the others.

16. "CQRS" is a command/query handler pattern

Kind: Doc mismatch · **Priority:** Low

What was found. Earlier documentation and code comments describe the Application layer as implementing CQRS. What exists is a **hand-rolled command/query handler pattern**: plain classes with a single `HandleAsync` method, registered in DI and injected straight into controllers. There is no MediatR or other mediator, no dispatcher, no pipeline behaviour, no separate read model, no separate write model and no event sourcing.

Impact. Terminology only — the pattern in use is clean and appropriate. But "CQRS" invites questions in a viva about read/write model separation that this codebase does not implement. The LLD uses the more precise description and states explicitly that no mediator library is present.

17. Unused injected dependency in `OfficerController`

Kind: Limitation · **Priority:** Low

What was found. `OfficerController` injects and assigns `GenerateRecommendationDraftCommandHandler_draftHandler`, but no method ever uses it. The controller uses `RecommendationOrchestratorAgent` for all recommendation generation.

Evidence. `src/Loan.Web/Controllers/OfficerController.cs` lines 19, 27, 34 — declaration, parameter, assignment; no further reference.

Impact. None functionally; a small unnecessary resolution per request. It is a useful clue that the officer path was migrated from the deterministic handler to the orchestrator without the old dependency being removed.

18. Compliance security events are illustrative, not real

Kind: Doc mismatch · **Priority:** Medium

What was found. The "Security Events" panel on the Compliance dashboard is built from two literal objects constructed on every page load:

```
new() { EventType = "PromptInjectionIntercept", SourceIp = "127.0.0.1",  
    QuerySnippet = "SYSTEM OVERRIDE: Ignore all previous instructions...",  
    TimestampUtc = DateTime.UtcNow.AddMinutes(-42) },  
new() { EventType = "UnauthorizedWriteAttempt", ... TimestampUtc = DateTime.UtcNow.AddMinutes(-85) }
```

Evidence. `src/Loan.Web/Controllers/ComplianceController.cs`, the `securityEvents` list.

Impact. The panel always shows exactly these two events with timestamps 42 and 85 minutes in the past, regardless of what has actually happened. Real prompt-injection interceptions are **not** recorded anywhere persistent — `PromptInjectionGuard` returns a refusal but writes no audit entry and increments no counter. `TridCompliantCount` is likewise set to the total application count rather than being assessed.

Important contrast. The **audit trail** on the same dashboard is genuine — it is assembled from real recommendation audit entries and real document override entries. Only the security events panel is illustrative.

Both documents describe the audit trail as real and note the security panel as demonstration content.

19. Identity threshold differs between two code paths

Kind: Limitation · Priority: Low

What was found. Two different confidence thresholds govern identity handling:

- `UploadAndExtractDocumentCommandHandler` auto-verifies identity when a `DriverLicenseOrPassport` document has all fields at 0.80 or above.
- The confirmation gate throughout the rest of the system — `ExtractedFieldRecord.NeedsConfirmation`, `DocumentAnalysisAgent`, `GenerateRecommendationDraftCommandHandler` — uses 0.85.

Impact. An identity document whose fields all score between 0.80 and 0.85 will set `IsIdentityVerified = true` while its individual fields simultaneously remain flagged as needing confirmation. The application is then both "identity verified" and "has unresolved fields", so the document gate still routes it to `PendingInformation`. The outcome is safe — the stricter gate wins — but the two thresholds are inconsistent and neither is named as a shared constant.

Related. The 0.85 threshold is repeated as a literal in at least five places rather than defined once.

20. Search filter values are string-interpolated

Kind: Security note · Priority: Low

What was found. OData filter expressions are composed by string interpolation without escaping:

```
filters.Add($"search.in(productId, '{targetProductId}', ALL', ',')");  
filters.Add($"policyVersion eq '{effectiveVersion}'");
```

Evidence. `src/Loan.Infrastructure/Search/AzureAiSearchPolicyRetriever.cs`.

Reachability assessment. Every current call site passes internal values, not free user text: `ComplianceReviewAgent` passes `application.ProductId` and `ProductRules.EffectiveVersion` (both from domain factory constants); the three chat endpoints pass no product or version at all; `McpToolServer.HandleSearchPolicyAsync` passes a caller-supplied `productId` — **the one externally influenced path** — but since the MCP endpoint is unauthenticated (observation 3), a crafted value could reach the filter. The impact ceiling is low: filter manipulation against a synthetic, read-only policy index containing no confidential data.

Conventional practice. Escape single quotes by doubling them, or validate against the known product identifier set. Noted as a pattern rather than an exploitable finding.

21. Unreachable `Views/Home/Index.cshtml`

Kind: Limitation · **Priority:** Low

What was found. `src/Loan.Web/Views/Home/Index.cshtml` and `Views/Home/Privacy.cshtml` exist, but `HomeController.Index` unconditionally redirects — to a role dashboard when authenticated, to `/Account/Login` otherwise — so `Index.cshtml` never renders. There is no `Privacy` action at all, so that view is unreachable by any route.

Impact. None functionally; dead view files only.

22. No deployment or CI/CD artefacts exist

Kind: Doc mismatch · **Priority:** Medium

What was found. A full repository search found none of the following: `Dockerfile`, `docker-compose.yml`, `*.bicep`, ARM templates, Terraform files, `.github/workflows/`, `azure-pipelines.yml`, or any publish profile.

Impact. The system runs locally only. The web application and the database are local; the AI, search and storage services it calls are genuinely in Azure and configured through User Secrets.

Correct way to describe it. "A locally-hosted ASP.NET Core application integrated with live Azure AI services." Not "a cloud-deployed solution" and not "an Azure-hosted application". The HLD's deployment section states this explicitly and labels its diagram as the current local topology.

D. What was verified as accurate

For balance, these significant claims were checked and hold up. They are the strong points of the implementation.

Claim	Verification
Zero-dependency domain	<code>Loan.Domain.csproj</code> contains no <code>PackageReference</code> element. The Clean Architecture claim is structurally enforced, not just asserted.
Dependency direction	Verified across all five project files: Application → Domain only; Infrastructure → Application + Domain; Web → Application + Infrastructure. No inward-pointing violation.
Deterministic financial maths	All DTI and LTV arithmetic is in <code>EligibilityCalculator</code> , pure C#, with no model involvement. Agent prompts explicitly forbid recalculation.
AI cannot approve or reject	Verified at four independent points: the draft handler's routing-state rejection, the MCP tool's pre-dispatch check, the orchestrator's deterministic override, and officer-only access to <code>OfficerDecisionCommandHandler</code> .
Genuine hybrid search	Both a text query and a <code>VectorizedQuery</code> are supplied in a single <code>SearchAsync</code> call — real hybrid retrieval, not vector-only relabelled.
Version-aware retrieval	OData filters on <code>policyVersion</code> and <code>isActive</code> are real, and the corpus deliberately contains an expired v1.0 and an active v2.0 of the same product guide to prove it.
Real MCP server	JSON-RPC 2.0 with <code>initialize</code> , <code>ping</code> , <code>tools/list</code> , <code>tools/call</code> , correct error codes, and a protocol version of <code>2024-11-05</code> . Not a mock.
Cross-application isolation	<code>ValidateApplicationScopeAsync</code> genuinely rejects mismatched synthetic identities, with a distinct error per failure mode. Covered by tests.
Prompt injection guard	16 keyword patterns plus 2 regular expressions, applied before any retrieval or model call, on all three assistants.
Verified-over-stated precedence	Implemented in <code>ApplicantFacts.EffectiveMonthlyIncome</code> and <code>EffectiveCreditScore</code> , and the calculator reads only the effective values.
Confidence gating	The 0.85 threshold genuinely blocks progression: unresolved fields force <code>PendingInformation</code> regardless of eligibility.
Zero-income guard	Zero or negative income yields a <code>null</code> DTI and <code>Ineligible</code> status — no division by zero. Covered by boundary tests.
Non-applicable LTV	Personal loans (no property valuation required) correctly yield <code>null</code> LTV treated as eligible, rather than a spurious failure.
Resilience behaviour	Retry with exponential backoff and jitter, 10-second timeout, circuit breaker with <code>Closed</code> / <code>Open</code> / <code>HalfOpen</code> , and caller cancellation correctly excluded from retry and from failure counting.
Safe degradation	Chat returns a degraded message; search returns empty and the assistant says it cannot verify — with an explicit log line confirming no synthetic fallback is used.
Blob fallback	Azure Blob Storage falls back to local file storage automatically and logs the decision.

Claim	Verification
File integrity	SHA-256 computed incrementally while streaming, stored with the document record and in blob metadata.
No committed secrets	Tracked configuration contains placeholders only. Real values live in User Secrets. Verified by inspecting every <code>appsettings*.json</code> .
Build and tests	<code>dotnet build</code> : 0 errors, 0 warnings. All six suites executed: 161 passed, 0 failed, 0 skipped — including live Azure OpenAI, Azure AI Search, Azure Blob Storage and SQL Server integration tests.

E. Suggested priority if you choose to act

No changes were made. If you want to address any of this before evaluation, this is the order I would suggest — cheapest and highest-value first.

Order	Item	Effort	Why first
1	#12 — MCP tool names in README	Minutes	A reader following the README gets three broken tool calls. Documentation-only fix.
2	#8 — test counts in <code>implementation-status.md</code>	Minutes	An evaluator may spot the 147-versus-161 mismatch and question the rigour of the rest.
3	#4 — frontmatter key casing	~8 lines	A genuine functional defect on a feature an administrator can demonstrate live.
4	#15 — <code>AzureAISearch</code> placeholder	Minutes	Makes the configuration requirement discoverable in a fresh clone.
5	#3 — authorise the MCP endpoint	Small	The most defensible security improvement, and an easy question to anticipate in a viva.
6	#5 — Admin tiles from <code>/health/details</code>	Small	Replaces the weakest-looking screen with something genuinely live.
7	#11 — pass application ID to <code>SaveAsync</code>	Small	Removes a latent defect currently masked only by call ordering.
8	#2 — Semantic Kernel: use it or remove it	Medium	Either choice is defensible; ambiguity is the problem.
9	#10, #17, #21 — remove dead code	Small	Tidying; reduces the chance of a confusing question.
10	#6 — add real OCR	Large	The most substantial capability gap, but properly a future enhancement rather than a fix.

A note on framing for your evaluation. Several items on this list are worth *mentioning* rather than hiding. Being able to say "extraction is LLM text-based, not OCR — here is exactly where I would add Document Intelligence" demonstrates better engineering judgement than presenting the system as complete. The same applies to the two recommendation paths and to the unauthenticated MCP endpoint: knowing your system's boundaries is a strength.

End of Implementation Observations and Discrepancies · Version 1.0 · 22 September 2026

No application code was modified in the production of this document or its companions.

Companion documents: `HLD-Loan-Application-and-Compliance-Review-Assistant.md` · `LLD-Loan-Application-and-Compliance-Review-Assistant.md`